

Water-monitor pipeline

How a drop of water becomes a number on the dashboard — from ESP32 pulse counting, through event detection, artifact screening, fixture labeling, and finally aggregation into totals.

Every step below names the responsible **file/function**, the **gates with their real thresholds**, the **DB columns written**, and an **"if it breaks here"** symptom so you can jump from a wrong number straight to the code that produced it.

Verified against the 0.3.1-dev14 working tree — committed through e9a846f (dev13: review verdicts + unusual-events fixes), plus the uncommitted dev14 rising-pressure phantom detector (rise_corr_backfill.py, migration 20260554). Re-check the constants against the code before trusting them.

Two invariants hold the whole thing together:

- **The leak alarm is firmware-side and reads live flow.** Nothing in the database — no split, zero, reprocess, or relabel — can mask a real leak.
- **Every litre that reaches a total passes through `apply_effective_volume()` (step 12).** The four feedback loops (hygiene auto-split, meter audit, your labels, recompute) all re-enter there rather than writing totals directly.

The spine at a glance

stage	what happens
1 ESP32 firmware	pulses / PPL -> L/min, pressure, onset flag
2 Home Assistant	live websocket + recorder history
3 Event starts?	flow >= floor 2s / pressure drop / both
4 Event open	flow + pressure buffers, propagation scan, ESP waveform
5 Event ends?	normal / recovery / flow-override / settled-noflow / 6h watchdog
6 Features	volume integral, signatures, active-flow metrics
7 Real water?	user-lock -> xtalk -> phantom (drop/rise) -> cross-talk -> dribble -> degraded (cap) -> sparse
9 Row stored	events table + hour ledger
10 Labeling ladder	softener -> washer -> dishwasher -> rules -> fingerprint -> k-NN -> composite
11 Anomaly score	vs frozen baseline
12 <code>apply_effective_volume()</code>	THE chokepoint — reverse old, post new
13 <code>hourly_volume</code>	signed deltas accumulate
14 Totals shown	daily summary, Water Use, dashboard, HA sensors, history

Side processes attach to the spine but sit outside it: the firmware leak alarm (a dead end), the HA recorder feeding backfill + meter audit, the ESP waveform path, the hygiene auto-split loop, the meter audit, and your labels. Each is described in its stage below.

Part 1 — Capture (raw water -> stored event)

1 · ESP32 senses

`firmware/esp-water-shut-off-3_12.yaml`

- **Flow chain:** pulse counter (40 Hz sampling) / `pp1_main` / `pp1_irr` (runtime NVS numbers, no /60) -> L/min -> clamp `>200` (ADC garbage) and `<0.01` (noise) to 0 -> 0.5 s window -> `water_usage1` / `water_usage2` at ~4 Hz.
- **Pressure chain:** `pressure_main` / `pressure_irr` at 1.375 s window, 2 Hz (recorded by HA — all backfill ever sees) and `pressure_*_fast` at 50 ms, 40 Hz (live only, never recorded).
- **Onset flag:** `flow_pulse_onset_*` ON at any pulse within 1 s, `delayed_off: 8s` so brief pauses don't split a draw.

If it breaks here — wrong PPL scales every volume by a constant factor; no pulses means nothing downstream ever fires. Check the HA `water_usage` sensor against a bucket test and the `pp1_*` entities.

Branch · Leak alarm (dead end by design)

`trickle_alert_main` / `trickle_alert_irr` fire when flow stays inside the min-max band for the threshold duration -> alert -> optional valve auto-close. Resets on any out-of-band flow. **Fully independent of the database** — it reads live flow, so no split/zero/reprocess/relabel below can ever mask a leak.

2 · Home Assistant relays

`app/ha_client.py` · `subscribe_entities`

A persistent websocket dispatches flow, fast pressure, and onset states to one `CircuitEventDetector` per circuit. HA's recorder separately stores 2 Hz pressure, onset transitions, flow, and the cumulative meter — the raw material replayed later by the importer (step 6) and the meter audit (step 12).

If it breaks here — an add-on restart wipes the in-progress event, so draws spanning a restart come back short or missing and are recovered only by the backfill importer. Look for restart timestamps in the add-on log near the gap.

3 · Does an event start?

`app/event_detector.py` · `CircuitEventDetector` — first trigger wins

trigger	condition
Flow	<code>flow >= MIN_FLOW_LPM</code> (per-circuit floor = 60 / PPL) sustained <code>FLOW_START_SECONDS = 2.0</code> , tolerating 2 sub-floor samples mid-ramp (<code>FLOW_START_DIP_TOLERANCE</code>)
Pressure	40 Hz fast-stream drop <code>>= threshold</code> , but only after the settled baseline has been stable <code>>= PRESSURE_STABLE_DURATION_S = 10</code> (blocks oscillation peaks)

trigger	condition
Pressure + flow	pressure opens the event first; flow arrival enriches it. <code>start_trigger</code> records which fired.

If it breaks here — missed small draws mean the floor is too high for the meter; phantom starts mean pressure oscillation is passing the 10 s stability gate. Compare event start times against raw `water_usage` history.

4 · While the event is open

[app/event_detector.py](#) · [RawEvent](#)

Two flow records accumulate in parallel: a 1 Hz `flow_readings` list (-> the 32-point signature) and timestamped `flow_samples` (-> the volume integral). A 400-sample pressure ring buffer (10 s @ 40 Hz) feeds the settled-baseline tracker (a 2 s window, 3 s back). The propagation scan searches up to 12 s before flow onset (`_PROP_MAX_LOOKBACK_S`), using a 0.10 psi noise band and 5 consecutive at-baseline samples to confirm the transient onset -> `propagation_delay_ms` .

Branch · ESP full-resolution waveform

`_WF_*` constants. Firmware can stream full-resolution waveform chunks (≤ 1500 samples each, ≤ 30 records per circuit, 2 h TTL). If assembly succeeds, `signature_source = 'esp_full_*` ' ; otherwise the software 1 Hz signature is the fallback.

5 · How does the event end?

[app/event_detector.py](#) — any one of five exits closes it

exit	condition
Normal	onset OFF and flow < <code>MIN_FLOW_LPM</code> , after a ≤ 120 s low-flow grace (<code>LOWFLOW_OFF_GRACE_S</code>) that coalesces fragments into one draw
Pressure recovery	dip recovered to $\leq 50\%$ of its magnitude (<code>PRESSURE_RECOVERY_FRACTION</code>) for 10 s
Flow override	pressure back at baseline for 5 min (<code>PRESSURE_RECOVERY_FLOW_OVERRIDE_S</code>) -> close even if flow reads stale-high (the old 27.6 h irrigation event)
Settled no-flow	flow zero the whole event + pressure settled 60 s (<code>SETTLED_NOFLOW_CLOSE_S</code>) -> close (a stuck event blocks every new event on the circuit)
Watchdog	hard force-close at 6 h (<code>MAX_EVENT_DURATION_S</code>)

If it breaks here — two draws merged into one points at the 120 s coalesce grace; a stretch of missed events points at something sitting open — grep the log for force-close / settled-no-flow lines.

Branch · Discarded (never becomes a row)

Volume < 1 mL (`MIN_EVENT_VOLUME_L`) is noise. A pressure-surge phantom — max pressure > baseline + 0.5 psi (`PRESSURE_SURGE_PHANTOM_PSI`) with no net drop — is a turbine artifact. Both are dropped.

6 · Features computed

[app/feature_extractor.py](#) · [FeatureExtractor](#)

- **Volume:** time integral of the timestamped samples (`flow_integral.integrate_litres`). A gap > 300 s marks `integration_quality = 'degraded'` (kept out of training).
- **Signatures:** 32-point flow + pressure envelopes (`flow_signature_json` , `pressure_signature_json`).
- **Hydraulics:** `pressure_delta_psi` , `pre_event_pressure_psi` , propagation delay, resistance shape.
- **Active-flow metrics** the verdicts depend on: `flow_integral_litres` , `flow_on_ratio` , `true_avg_flow_lpm` , `active_flow_segment_count` .
- **Time features:** `hour_sin` / `hour_cos` , day-of-week, weekend.

If it breaks here — event volume disagreeing with the meter -> check `integration_quality` and `volume_recorder_litres` for the firmware-meter cross-check.

Branch · Backfill importer joins here

[app/historical_importer.py](#)

- **Triggers:** startup (back to `MAX_BACKFILL_DAYS`), periodic catch-up from `import_state.last_check_ts` , or a manual range.
- **Sources:** `flow_pulse_onset` ON/OFF transitions first (bridging gaps < 15 s of turbine chatter), then a sustained-flow-threshold fallback.
- **Duplicate gate:** a period overlapping an existing event by >=30 s (or >=10 s and >=80% of the shorter) is skipped — safe to re-run.
- **Irrigation cross-talk post-pass:** flags main-circuit events overlapping irrigation when the pressure-swing ratio (irrigation / main) >= 1.3, with the frozen <=1.5 L cap -> `match_rejection_reason = 'irrigation_cross_talk'` .
- Rebuilt events always carry `start_trigger = 'flow'` (no 40 Hz pressure history to reconstruct).

If it breaks here — duplicated or still-missing backfill -> check `import_state` and whether the gap's HA recorder history actually exists.

Part 2 — The verdict cascade (is it real water?)

[app/feature_extractor.py](#) · `_finalize_derived_verdicts()` — first match wins

#	check	condition	result
7a	You already classified it	<code>user_classified = 1</code>	your verdict stands; auto-detection never re-flags
7b	Durable irrigation cross-talk	<code>mrr = 'irrigation_cross_talk'</code> (from importer)	<code>veff = 0</code> ; survives every reprocess unless you relabel it real

#	check	condition	result
7c	Phantom (pressure restoration)	<code>dur >= 120 s (</code> <code>_PHANTOM_NOFLOW_MIN_DURATION_S</code> ; a legacy no-metrics event needs the 30-min floor) and <code>dP < 2.0 psi</code> (frozen) and <code>flow_integral < 1.0 L</code> and <code>flow_on_ratio < 0.05</code> . Rescue: true avg flow <code>>= 2.0 L/min</code> -> real brief draw	<code>is_pressure_restoration_phantom=1</code> , <code>veff=0</code> , excluded
7d	Rising-pressure phantom (dev14)	the OPPOSITE of 7c — flow tracked a pressure rise (turbine spun on climbing city pressure, not demand). Fires when flow-pressure correlation <code>flow_pressure_corr >= 0.6</code> and <code>0 < vol < 1.0 L</code> (strict, frozen) and <code>dur <= 120 s</code> . <code>corr = None</code> -> never fires; skips degraded/user-typed	reuses <code>is_pressure_restoration_phantom=1</code> , <code>veff=0</code> ; distinct <code>mrr='rising_pressure_phantom'</code> ; excluded
7e	Cross-talk (other circuit)	same no-flow ceilings (<code>flow_integral < 1.0 L</code> , <code>flow_on_ratio < 0.05</code>), but <code>dP >= 2.0 psi</code> and <code>dur >= 120 s (</code> <code>_XTALK_MIN_DURATION_S</code> , calib-overridable). The dP floor separates it from 7c	<code>is_cross_talk=1</code> , <code>veff=0</code> , excluded
7f	Dribble (sensor trickle)	<code>vol < 0.5 L</code> and avg flow <code>< 1.0 L/min</code> and <code>dP < 1.5 psi</code> . Guard: coarse meter (60/PPL floor <code>>= 0.5 L/min</code>) measures low flow reliably -> never dribble	<code>is_low_flow_dribble=1</code> , <code>veff=0</code> , excluded
7g	Degraded supply (pulsing)	<code>degraded_supply = 1</code> — flow reading unreliable	<code>veff =</code> <code>volume_litres_estimated</code> , capped (see below), excluded
7h	Sparse envelope	<code>dur >= 10 min (</code> <code>_SPARSE_ENVELOPE_MIN_DURATION_S = 600 s</code>) and flow on <code><= 10%</code> of it	litres kept , <code>mrr='sparse_envelope'</code> , excluded; targeted by hygiene loop
8	Real use	none of the above	<code>volume_litres_effective = volume_litres</code> , method 'raw' , counts + trains

Envelope cap (dev10, guards 7f). A degraded/pulsing envelope estimate can badly over-read, so `_cap_envelope_estimate()` limits it to `max(1.5 x flow_integral_litres, 2.0 L)` (`_ENVELOPE_CAP_FLOW_MULT = 1.5` , `_ENVELOPE_CAP_FLOOR_L = 2.0`). The uncapped value + cap base are recorded in `degraded_diagnostic_json` for audit.

Suppression-averted backstop (dev10, wraps 7c). A leak-safety guard: when the phantom rule would fire but the event actually measured `volume_litres >= 10 L` (`_PHANTOM_REVIEW_FLAG_LITRES = 10.0`), the phantom is averted — volume **kept, not zeroed** — and the event is surfaced for review (`phantom_suppression_averted = 1` , `anomaly_type = 'suppression_averted'` , `flagged = 1` ; still excluded from training). Silently zeroing 10 L+ is riskier than showing a review draw. Migration 20260551 restored already-zeroed large phantoms through the ledger.

Where `flow_pressure_corr` comes from (7d). Computed live from the event's waveform (`_flow_pressure_correlation` over the full flow + pressure series) and stored on the row (column added by migration 20260554). Stored 32-point signatures CANNOT reconstruct it (the pressure signature clamps drop-fraction to [0,1], erasing the above-baseline rises), so old events are handled by a one-shot backfill worker (branch below). The verdict is re-applied to any event with a stored corr by `reprocess_rising_pressure_phantoms()` , part of the same repair chain as the phantom/cross-talk/dribble passes.

If it breaks here — real water zeroed (or noise surviving to step 8) -> read `volume_estimation_method` and the three `is_*` flags. A big draw flagged for review instead of counted-clean -> `phantom_suppression_averted` . A small burst zeroed during a pressure rise -> `flow_pressure_corr` (≥ 0.6 fired it). Relabeling re-runs the cascade with your label.

Branch · Rising-corr backfill (dev14, one-shot)

[app/rise_corr_backfill.py](#)

Events predating the `flow_pressure_corr` column can't have it re-derived from stored signatures, so this worker re-fetches each candidate's flow + pressure history from the HA recorder, resamples onto the 1 Hz grid, runs the same `_flow_pressure_correlation` , and applies the verdict through the shared repair pass. Candidates are only events the live detector could have caught: ≤ 120 s, $0 < \text{vol} < 1$ L, unlabeled, unflagged, non-degraded, no artifact verdict, no stored corr. Sweeps oldest-first; a clean sweep sets `rise_corr_backfill_done = 1` and never runs again (events whose history is gone stay counted — leak-safe).

9 · Row stored + hour ledger posted

[app/database.py](#) · `upsert_event_and_apply_hourly_volume()`

One row in `events` (volumes raw + effective, signatures, artifact flags, provenance) is written, and the event's litres are posted to its hour bucket via the chokepoint (step 12) in the same step. Sequence context (`seconds_since_prev_event` , `cycle_pulse_count`) is filled in.

Branch · Hygiene auto-split loop

[app/reprocess.py](#)

- **Candidates:** unlabeled events whose actual flow is $< 25\%$ of their span (incl. inflated `sparse_envelope` singles).
- **Dry-run:** the importer counts real draws inside; if 2–10 are found ->
- **Atomic** `reprocess_window()` : delete -> auto-widen to engulf the deleted spans -> re-import -> restore the originals on any failure (all-or-nothing, no lost water).
- **Never touches** user-labeled, artifact-flagged, anomaly-flagged, or softener-session events. Loops back through step 6.

Part 3 — Labeling (which fixture?)

[app/database.py](#) · `reclassify_all_events_from_signatures()` – a ladder; first rung to claim wins

run g	detector	gates	stamps
10a	Softener session	skipped unless <code>has_water_softener</code> + right circuit. Low-flow (≤ 1.5 L/min, peaks < 4) at regen time +20 min (local); chains gaps ≤ 45 min, session ≤ 210 min; needs backwash ≥ 30 L and brine span ≥ 90 min	<code>water_softener</code> <code>softener_session</code>
10b	Washer cycle	anchor fill ≥ 9 L, 80–400 s, peak 7.5–15 L/min; siblings 0.8–1.3x anchor peak, 2–45 min away, ≥ 0.5 L, ≤ 400 s, not flush; needs anchor + ≥ 2 siblings. Live retro-stamps trailing 50 min	<code>washing_machine</code> <code>washer_cycle</code>
10c	Dishwasher cycle	≥ 3 chained gentle fills: 0.2–3.5 L, peak ≤ 3.6 L/min, not flush; fills ≤ 30 min apart, run ≤ 180 min; skips artifacts + washer/softener members	<code>dishwasher</code> <code>dishwasher_cycle</code>
10d	Per-event rules	first hit wins — see below	<code>fixture</code> , <code>rule_*</code> / <code>zone_default</code>
10e	Fingerprint tier	only if rules abstain, before k-NN — see below	<code>fixture</code> , <code>fingerprint</code>
10f	k-NN residual	only if the fingerprint tier also abstains — see below	<code>fixture</code> , <code>knn</code>
10g	Composite	sustained ≥ 300 s + usable waveform (≥ 30 bins, ≤ 15 s/bin); embedded toilet = 3–8 L excess at ≥ 3 L/min over a rolling 35th-pct baseline	unlabeled -> other <code>composite</code> ; writes <code>embedded_fixtures_json</code>

10d · Per-event rules (first hit wins)

- **Toilet:** 2.2–8.5 L, 20–150 s, peak ≥ 5 L/min, **and** (pressure transient or dP ≥ 1.5 psi).
- **Dishwasher (single):** 0.2–3.5 L, peak ≤ 3.6 L/min, `cycle_pulse_count` ≥ 3 .
- **Shower:** ≥ 30 L and ≥ 300 s and peak ≥ 6 L/min — or 15–30 L and ≥ 240 s.
- **Zone (zone circuits only):** ≥ 240 s and peak ≥ 5 L/min.

Bands are per-home calibrated once at activation (`rule_calibration.py` : weighted-percentile fit, capped at ≤ 2 x the default span, do-no-harm k-fold — a fit that regresses recall is discarded). A PPL change triggers partial recalibration of artifact thresholds only, never these rule bands.

10e · Fingerprint tier (dev11, fingerprint_matcher.py)

A whole-waveform nearest-neighbour match — stronger evidence than k-NN's scalar summaries, so it runs ahead of k-NN and short-circuits it on a hit.

- **Fingerprint:** the event's un-normalised waveform trio — absolute-time flow (L/min), cumulative volume (L), pressure drop below baseline (psi) — on a 4 s grid, 64 cells (first 256 s). Built only when the waveform supports it: ≥ 10 s, ≥ 4 bins, peak ≥ 0.3 L/min.
- **Library:** user-labeled events only (`user` / `training` / `cycle` sources). Fingerprint labels never join the library -> no chaining, no drift.
- **Gates:** ≥ 10 library labels total (`MIN_LIBRARY_N`), ≥ 5 per class (`MIN_CLASS_LIBRARY`). Accept distance is self-calibrating — a percentile of the library's own NN-distance distribution: 30th once mature (≥ 100 labels), tightened to the 15th below that.

- **CYCLE_ONLY exception:** unlike k-NN, this tier may inherit `washing_machine` / `dishwasher` — a full-waveform match against a user-confirmed example is trusted (83/83 dishwashers correct).
- Measured here: ~30% coverage at ~97% precision; ~29% of pipeline-declined events at ~94%.
Per-circuit 5-min library cache, invalidated when you save a label.

10f · k-NN residual

- **Pool:** your labeled events with `excluded_from_training = 0` ; needs ≥ 10 labels total and ≥ 2 per class.
- **Distance:** the active-flow scale set (preferred once backfilled) — log-scaled features weighted volume 1.52, dP 2.88, duration 1.40, flows 0.70/0.74, `flow_on_ratio` 0.25, `cycle_pulse_count` 0.75, `hour_sin` / `hour_cos` 0.35; a pre-backfill event uses the legacy set.
- **Vote:** $k=5$, inverse-distance weighted, capped at 4 neighbors per class.
- **Abstains when:** total score < 1.5 (out-of-distribution), winner share < 0.6 (ambiguous), or the winner is `other` .
- **CYCLE_ONLY guard:** a lone k-NN vote can never stamp `washing_machine` / `dishwasher` / `water_softener` — only their cycle detectors (or the fingerprint tier above) may. A suppressed real member is re-stamped by its detector on the next sweep.

If it breaks here — a wrong fixture name -> read `matched_via` first; it names the exact rung that claimed the event, so you know which thresholds to compare against volume / duration / peak.

11 · Anomaly score + surfacing (every event)

[app/anomaly_baseline.py](#) · [alert_manager.py](#) · [routers/history.py](#)

Scored against the frozen per-home baseline (fit at activation, never online-adapted): volume, type, and time-of-day pattern -> `anomaly_score` / `anomaly_type` , setting `flagged = 1` . Core scoring is unchanged, but dev9 wired up the surfacing that was previously dead:

- **Suppression-averted override:** a `phantom_suppression_averted` event (step 7c backstop) is forced anomalous before the artifact gate, so an excluded big draw still shows for review.
- ``triggered_alert`` is now stamped 1 when `alert_manager.fire()` actually notifies — an audit trail of "this event alerted" (previously never populated).
- **History surfacing:** a `?filter=anomaly` view lists every `flagged` event (bypassing the hide-not-real toggle); a display-time `anomaly_reason` splits the flag into badges — `review_draw` (suppression-averted), `estimated` (degraded/envelope), `high_usage` (rest). Marking reviewed sets `user_reviewed = 1` and clears the dashboard unreviewed count.
- **Review verdicts (dev13):** `review_verdict` (migration 20260553) is `normal` (confirmed legit) or `unknown` (looked, didn't recognise it); both stamp `user_reviewed = 1` . Display + training-workflow only — never touches `volume_litres_effective` or the live frozen-baseline score. Its one job: `fit_usage_baselines()` holds `unknown` events OUT of the next baseline refit, so an unidentified draw can't stretch an envelope toward normal. Labeling clears `unknown` .
- **Unusual-events fix (dev12):** the anomaly filter ran in Python AFTER the newest-100 fetch, so flagged events older than the 100th row vanished from the view (card said 95, list showed 2). `flagged_only` / `degraded_only` now run inside the SQL WHERE, so the card count matches the list.

Part 4 — Totals (events -> the numbers you see)

12 · The volume chokepoint

[app/database.py](#) · [apply_effective_volume\(\)](#)

The only path by which any event's litres reach totals:

- If a prior posting exists (`hourly_volume_applied_litres` / `_bucket`), post a **reverse delta** to the old hour bucket.
- If the new effective volume > 0, post it to the hour of `start_ts` . If it's 0, the bucket is NULL — the event contributes nowhere.

Callers (the complete list): live insert (step 9), `volume_recompute.py` , `recorder_reconcile.py` , the low-flow coalescer, and relabel/reprocess paths. **Invariant:** `volume_ledger_discrepancy()` ~ 0 (sum of applied amounts = sum of `hourly_volume`).

If it breaks here — totals drifting from the sum of events means a writer bypassed this function — run the discrepancy check first.

Branch · Meter audit (the reconciliation loop)

[app/recorder_reconcile.py](#) · [reconcile_circuit_volumes\(\)](#)

Hourly, over settled events (> 20 min old, <= 6 h horizon), **healthy events only** — never phantom, cross-talk, dribble, degraded, or anything you classified. Computes the firmware cumulative meter's delta across the event and stores it as `volume_recorder_litres` . **Auto-corrects** only when recorder samples bracket the event edges within +-2 min and the divergence is > 0.5 L and > 20% of the larger value — routing the fix through `apply_effective_volume()` . Otherwise it just flags a backlog you can apply manually.

If it breaks here — per-event volumes disagreeing with the physical meter -> compare `volume_litres_effective` vs `volume_recorder_litres` , then check `reconcile_state` for whether that window was reconciled.

13 · Hour buckets accumulate

`hourly_volume` table

Signed deltas accumulate via `INSERT ... ON CONFLICT DO UPDATE SET volume_litres = volume_litres + excluded.volume_litres` . This table is already net of every zeroing, correction, split, and relabel above, and is what the 24 h dashboard chart reads.

Branch · Your labels (the feedback loop)

[app/routers/history.py](#) · [patch_event\(\)](#)

Saving a label writes `user_fixture_type` + `user_classified` = 1 — never overwritten by any machine pass. Appliance labels propagate to cycle-mates (+-45 min, tagged `fixture_label_source` = 'cycle'). 8 s after the last edit a background reclassify re-runs the ladder over unlabeled events (skipped once the baseline is locked). An hourly maturity recheck re-runs events < 6 h old so provisional labels settle once cycle-mates exist. Any volume change re-enters at step 12.

If it breaks here — a label that "won't stick" usually means you're looking at a different event (a split/reprocess created new IDs), or the maturity recheck re-stamped an unlabeled mate — your own labeled row is never touched.

14 · What you see

surface	file / function	how the total is computed
Daily summary	<code>compute_daily_summary()</code>	<code>SUM(COALESCE(volume_litres_effective, volume_litres, 0))</code> per circuit-day; phantoms excluded from the event count only (litres already 0)
Water Use page	<code>get_category_rollup()</code>	groups by effective type — precedence <code>user_fixture_type</code> -> <code>confirmed_fixture</code> -> <code>matched_fixture_type</code> -> <code>cluster hint</code> -> <code>other</code> ; <code>WHERE is_pressure_restoration_phantom = 0</code> ; lifetime + windowed sums
Dashboard + HA	<code>dashboard.py</code> , <code>fixture_publisher.py</code>	24 h chart from hour buckets; per-fixture + per-category totals published to HA as <code>total_increasing</code> sensors
History page	<code>routers/history.py</code>	reads events rows directly; the hide-not-real toggle filters rows at render time only — totals never change, litres were decided at step 7

Quick debugging index

symptom	start at	first thing to check
A total looks wrong	Part 4	the event's <code>volume_litres_effective</code> vs <code>volume_litres</code> , then <code>volume_estimation_method</code>
Totals != sum of events	step 12	<code>volume_ledger_discrepancy()</code> — a writer bypassed the chokepoint
Wrong fixture name	Part 3	<code>matched_via</code> — names the exact ladder rung that claimed it
Event volume != physical meter	step 12 branch	<code>volume_litres_effective</code> vs <code>volume_recorder_litres</code> , then <code>reconcile_state</code>
Event missing / merged / split	Part 1	restart timestamps (websocket), the 120 s coalesce grace, or force-close lines (stuck event)
Real water shows as 0 L	step 7	the three <code>is_*</code> flags + <code>volume_estimation_method</code>
Small burst zeroed during a pressure rise	step 7d	<code>flow_pressure_corr</code> <code>>= 0.6</code> fired the rising-pressure phantom
Big draw flagged 'review' not counted-clean	step 7c backstop	<code>phantom_suppression_averted</code> (<code>>=10 L</code> phantom kept + flagged on purpose)
Degraded volume looks capped/too low	step 7f	<code>degraded_diagnostic_json</code> (<code>envelope_cap_applied</code> , <code>envelope_uncapped_litres</code>)

symptom	start at	first thing to check
Label came from fingerprint and looks wrong	step 10e	library size / self-calibrated threshold; save a correct label to re-seed
A label won't stick	step 13 branch	whether the ID changed (reprocess) — user rows are never overwritten